# Python 程式語言

## Lecture 8: GUI Programming with Tkinter and PyQt

Instructor: 顏家珩 Chia-Heng Yen

# Course Roadmap

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Basic Syntax and Flow Control
  - Data Structures and Collections
- Advanced Programming
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - Algorithm Implementation in Python
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Data Analysis Fundamentals
  - Scientific Computing Applications
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

# Course Objectives

- Understand GUI programming fundamentals
- Create interactive desktop applications using Tkinter
- Master layout management and event handling
- Explore PyQt/PySide for advanced GUI development

# Outline

- Introduction to GUI Programming
- Tkinter Fundamentals
- Tkinter Advanced Topics
- PyQt/PySide Introduction

# Introduction to GUI Programming

# What is GUI?

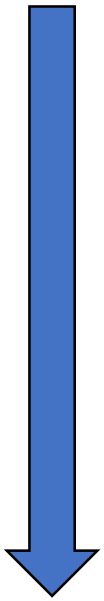
- GUI = Graphical User Interface
- Key Concepts
  - Visual Elements: Windows, buttons, menus, etc.
  - Event-Driven: Responds to user actions
  - Interactive: Real-time user feedback

- Command Line Interface (CLI)

  $ python calculator.py
  Enter operation: +
  Enter numbers: 5 3
  Result: 8

- Graphical User Interface (GUI)

  [Window with buttons]
  Click buttons to calculate
  Visual feedback immediately



# Why Learn GUI Programming?

- Advantages
  - User-Friendly
  - Intuitive interaction
  - Visual feedback
  - Lower learning curve
- Professional Applications
  - Desktop software development
  - Data visualization tools
  - Configuration interfaces
- Practical Skills
  - Expand programming toolkit
  - Create user-facing applications

# Python GUI Frameworks

- Popular Choice

| Framework | Pros | Cons | Best For |
|---|---|---|---|
| Tkinter | Built-in, simple | Basic appearance | Quick prototypes |
| PyQt/PySide | Professional, powerful | Complex, licensing | Enterprise apps |
| wxPython | Native look | Larger size | Cross-platform apps |
| Kivy | Mobile support | Different paradigm | Mobile/Touch apps |

- This Course Focus
  - Tkinter (Foundation)
  - PyQt/PySide (Advanced)

# Tkinter Fundamentals

# What is Tkinter?

- Python's Standard GUI Library
- Key Features
  - Built-in: No installation needed
  - Cross-platform: Works on Windows, Mac, Linux
  - Lightweight: Small footprint
  - Well-documented: Extensive resources

```
# Import Tkinter
import tkinter as tk
from tkinter import ttk                 # Themed widgets

# Check installation
print(tk.TkVersion)                     # Shows version
```

# Your First Tkinter Window

- Basic Window Structure

```
import tkinter as tk

# Create main window
root = tk.Tk()

# Set window properties
root.title("My First GUI")          # Window title
root.geometry("400x300")            # Width x Height
root.resizable(True, True)          # Can resize

# Start event loop
root.mainloop()
```

# Your First Tkinter Window

- Window Methods

```
# Common window operations
root.title("Application Name")          # Set title
root.geometry("800x600")                # Set size
root.geometry("800x600+100+50")         # Size + position
root.minsize(400, 300)                  # Minimum size
root.maxsize(1200, 800)                 # Maximum size
root.iconbitmap("icon.ico")             # Set icon
root.configure(bg="white")              # Background color
```

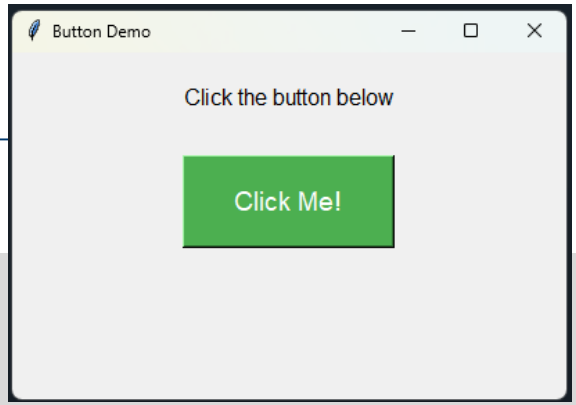
# Basic Widgets

- Label Widget

```
import tkinter as tk

root = tk.Tk()
root.title("Label Demo")
root.geometry("400x300")

# Create label
label1 = tk.Label(
        root,
        text="Welcome to Python GUI!",      # Text content
        font=("Arial", 16, "bold"),         # Font style
        fg="blue",                          # Foreground color
        bg="lightyellow",                   # Background color
        padx=20,                            # Horizontal padding
        pady=10                             # Vertical padding
)
label1.pack()                               # Display widget

# Label with image
# photo = tk.PhotoImage(file="image.png")
# label2 = tk.Label(root, image=photo)
# label2.pack()

root.mainloop()
```

# Button Widget

- Interactive Buttons



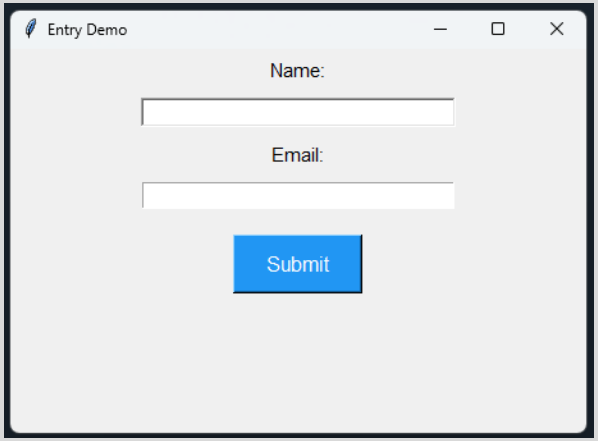

```
import tkinter as tk

def on_button_click():
"""Function called when button is clicked"""
        print("Button clicked!")
        label.config(text="Button was clicked!")

root = tk.Tk()
root.title("Button Demo")
root.geometry("400x250")

# Status label
label = tk.Label(
        root,
        text="Click the button below",
        font=("Arial", 12)
)
label.pack(pady=20)
```

```
# Create button
button = tk.Button(
        root,
        text="Click Me!",          # Button text
        command=on_button_click,
        # Function to call
        font=("Arial", 14),
        bg="#4CAF50",              # Green background
        fg="white",                # White text
        padx=30,
        pady=15,
        relief=tk.RAISED,          # Button style
        cursor="hand2"             # Cursor type
)
button.pack(pady=10)

root.mainloop()
```

# Entry Widget

- Text Input Fields

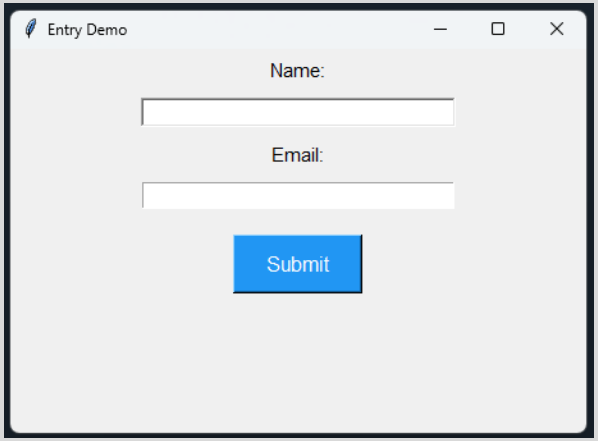
```
import tkinter as tk

def submit_data():
        """Get input from entry widgets"""
        name = name_entry.get()
        email = email_entry.get()

        result_label config( text=f"Name: {name}\nEmail: {email}" )
        # Clear entries
        name_entry.delete(0, tk.END)
        email_entry.delete(0, tk.END)

root = tk.Tk()
root.title("Entry Demo")
root.geometry("450x300")
# Name input
tk.Label(root, text="Name:", font=("Arial", 11)).pack(pady=5)
name_entry = tk.Entry(
        root,
        font=("Arial", 11),
        width=30,
        bd=2,                           # Border width
        relief=tk.SUNKEN                # Border style
)
name_entry.pack(pady=5)
```



# Entry Widget

- Text Input Fields

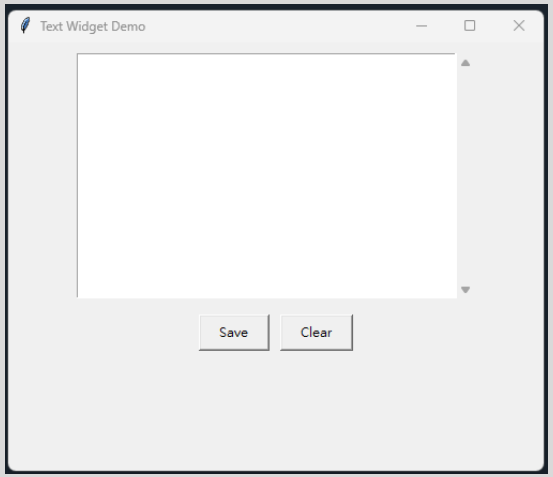
```
# Email input
tk.Label(root, text="Email:", font=("Arial", 11)).pack(pady=5)
email_entry = tk.Entry(root, font=("Arial", 11), width=30)
email_entry.pack(pady=5)

# Submit button
submit_btn = tk.Button(
        root,
        text="Submit",
        command=submit_data,
        font=("Arial", 12),
        bg="#2196F3",
        fg="white",
        padx=20,
        pady=8
)
submit_btn.pack(pady=15)
# Result display
result_label = tk.Label(
        root,
        text="",
        font=("Arial", 10),
        fg="green"
)
result_label.pack(pady=10)
root.mainloop()
```



# Text Widget

- Multi-line Text Input

```
import tkinter as tk from tkinter
import scrolledtext

def save_text():
        """Get text from Text widget"""
        content = text_area.get("1.0", tk.END) # Get all text
        print(content)
        status_label config(text="Text saved!")

def clear_text():
        """Clear text area"""“
        text_area.delete("1.0", tk.END)
        status_label config(text="Text cleared!")

root = tk.Tk()
root.title("Text Widget Demo")
root.geometry("500x400")
# Text area with scrollbar
text_area = scrolledtext.ScrolledText(
        root,
        width=50,
        height=15,
        font=("Consolas", 10),
        wrap=tk.WORD                    # Word wrap
)
text_area.pack(pady=10, padx=10)
```

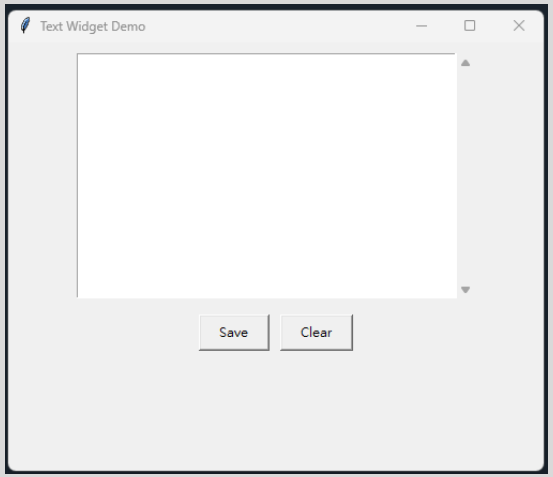


# Text Widget

- Multi-line Text Input

```
# Button frame
button_frame = tk.Frame(root)
button_frame.pack(pady=5)

# Save button
save_btn = tk.Button(
        button_frame,
        text="Save",
        command=save_text,
        padx=15,
        pady=5
)
save_btn.pack(side=tk.LEFT, padx=5)

# Clear button
clear_btn = tk.Button(
        button_frame,
        text="Clear",
        command=clear_text,
        padx=15,
        pady=5
)
clear_btn.pack(side=tk.LEFT, padx=5)
```

```
# Status label
status_label = tk.Label(root, text="", fg="blue")
status_label.pack(pady=5)

root.mainloop()
```



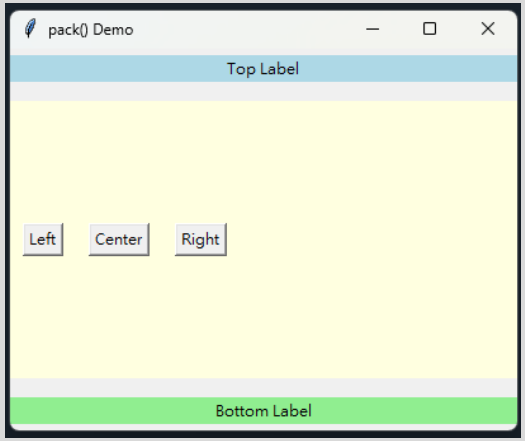
# Layout Management

- Three Layout Managers
  1. pack() - Simplest
     - Stacks widgets vertically or horizontally
     - Good for simple layouts
  2. grid() - Table-like
     - Arranges widgets in rows and columns
     - Best for forms and structured layouts
  3. place() - Absolute positioning
     - Precise control over widget position
     - Less flexible, rarely used

# pack() Layout Manager

- Vertical and Horizontal Packing

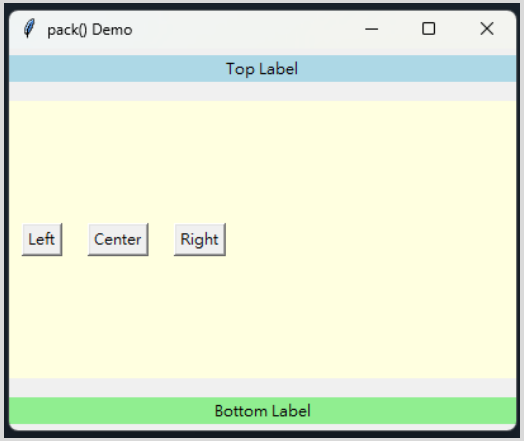
```
import tkinter as tk

root = tk.Tk()
root.title("pack() Demo")
root.geometry("400x300")

# Vertical packing (default)
tk.Label(root, text="Top Label", bg="lightblue").pack(
        side=tk.TOP,            # Position
        fill=tk.X,              # Fill horizontally
        pady=5                  # Vertical spacing
)

tk.Label(root, text="Bottom Label", bg="lightgreen").pack(
        side=tk.BOTTOM,
        fill=tk.X,
        pady=5
)
```



# pack() Layout Manager

- Vertical and Horizontal Packing

```
# Horizontal packing
button_frame = tk.Frame(root, bg="lightyellow")
button_frame.pack(side=tk.TOP, fill=tk.BOTH, expand=True, pady=10)

tk.Button(button_frame, text="Left").pack(
        side=tk.LEFT,
        padx=10
)

tk.Button(button_frame, text="Center").pack(
        side=tk.LEFT,
        padx=10
)

tk.Button(button_frame, text="Right").pack(
        side=tk.LEFT,
        padx=10
)

root.mainloop()
```



# pack() Layout Manager

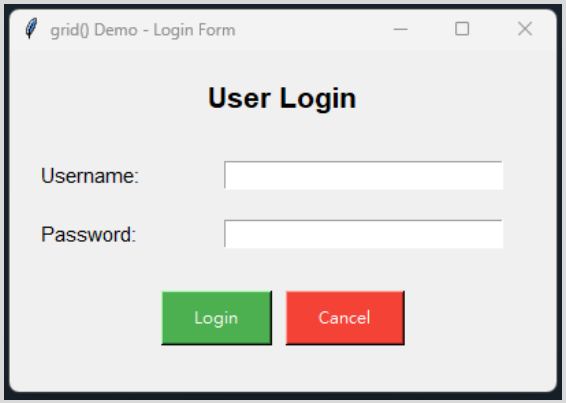
- pack() Parameters

```
widget.pack(
        side=tk.TOP,            # TOP, BOTTOM, LEFT, RIGHT
        fill=tk.NONE,           # NONE, X, Y, BOTH
        expand=False,           # Expand to fill space
        padx=0,                 # External horizontal padding
        pady=0,                 # External vertical padding
        ipadx=0,                # Internal horizontal padding
        ipady=0,                # Internal vertical padding
        anchor=tk.CENTER        # N, S, E, W, NE, NW, SE, SW, CENTER
)
```

# grid() Layout Manager

- Row and Column Layout

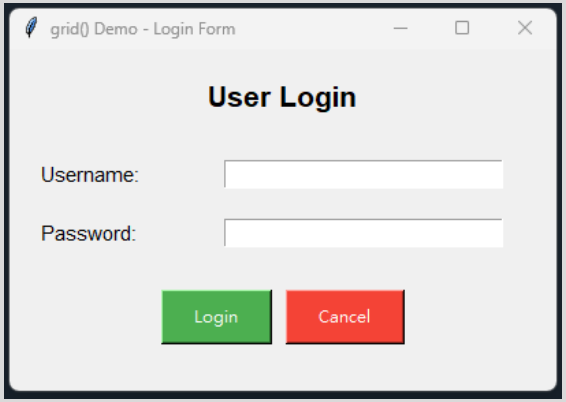
```
import tkinter as tk

root = tk.Tk()
root.title("grid() Demo - Login Form")
root.geometry("400x250")

# Configure grid weights
root.columnconfigure(1, weight=1)

# Title
title_label = tk.Label(
            root,
            text="User Login",
            font=("Arial", 16, "bold")
)
title_label.grid(
            row=0, # Row number
            column=0, # Column number
            columnspan=2, # Span across columns
            pady=20
)
```



# grid() Layout Manager

- Row and Column Layout

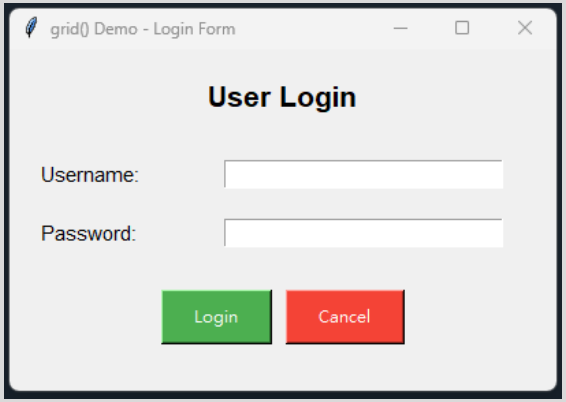
```
# Username
tk.Label(root, text="Username:", font=("Arial", 11)).grid(
            row=1,
            column=0,
            sticky=tk.W,            # Align west (left)
            padx=20,
            pady=10
)
username_entry = tk.Entry(root, font=("Arial", 11), width=25)
username_entry.grid(row=1, column=1, padx=20, pady=10)

# Password
tk.Label(root, text="Password:", font=("Arial", 11)).grid(
            row=2,
            column=0,
            sticky=tk.W,
            padx=20,
            pady=10
)
password_entry = tk.Entry(root, show="*", font=("Arial", 11), width=25)
password_entry.grid(row=2, column=1, padx=20, pady=10)
```



# grid() Layout Manager

- Row and Column Layout

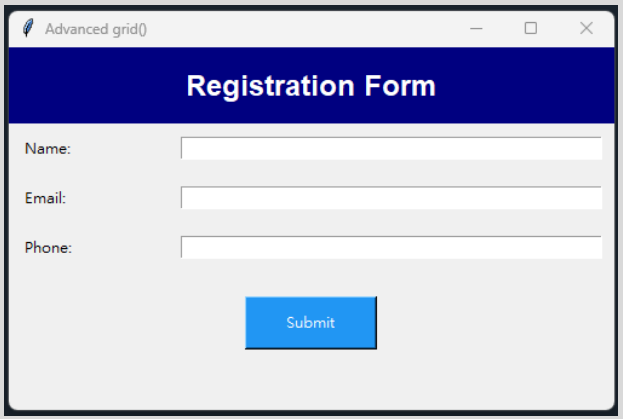
```
# Buttons
button_frame = tk.Frame(root)
button_frame.grid(row=3, column=0, columnspan=2, pady=20)

login_btn = tk.Button(
            button_frame,
            text="Login",
            bg="#4CAF50",
            fg="white",
            padx=20,
            pady=8
)
login_btn.pack(side=tk.LEFT, padx=5)
cancel_btn = tk.Button(
            button_frame,
            text="Cancel",
            bg="#f44336",
            fg="white",
            padx=20,
            pady=8 )
cancel_btn.pack(side=tk.LEFT, padx=5)

root.mainloop()
```



# grid() Advanced Features

- Spanning and Sticky Options

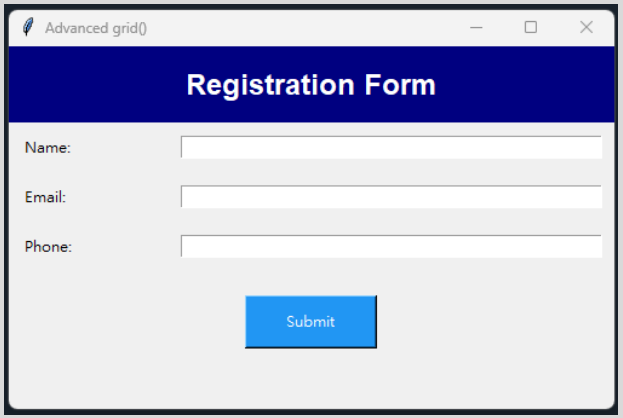
```
import tkinter as tk

root = tk.Tk()
root.title("Advanced grid()")
root.geometry("500x300")

# Configure column weights
for i in range(3):
        root.columnconfigure(i, weight=1)

# Header spanning multiple columns
header = tk.Label(
        root,
        text="Registration Form",
        font=("Arial", 18, "bold"),
        bg="navy",
        fg="white",
        pady=15
)
header.grid(
        row=0,
        column=0,
        columnspan=3,        # Span 3 columns
        sticky="ew"          # Stretch east-west
)
```



# grid() Advanced Features

- Spanning and Sticky Options

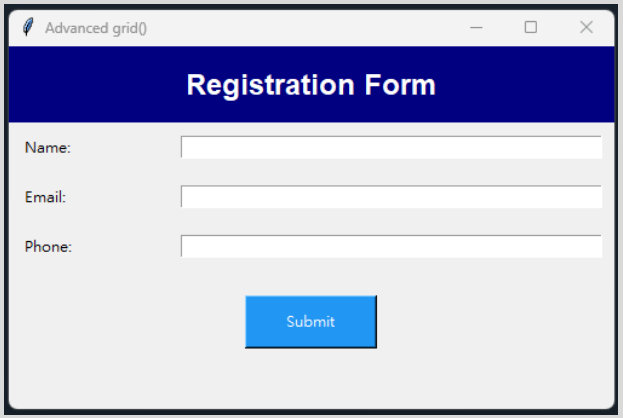
```
# Form fields
labels = ["Name:", "Email:", "Phone:"]
for i, label_text in enumerate(labels, start=1):
        tk.Label(root, text=label_text).grid(
                row=i,
                column=0,
                sticky=tk.W,
                padx=10,
                pady=10
        )
        tk.Entry(root, width=30).grid(
                row=i,
                column=1,
                columnspan=2,
                sticky="ew",
                padx=10,
                pady=10
        )
```



# grid() Advanced Features

- Spanning and Sticky Options

```
# Submit button
submit_btn = tk.Button(
            root,
            text="Submit",
            bg="#2196F3",
            fg="white",
            padx=30,
            pady=10
)

submit_btn.grid(
            row=4,
            column=0,
            columnspan=3,
            pady=20
)

root.mainloop()
```



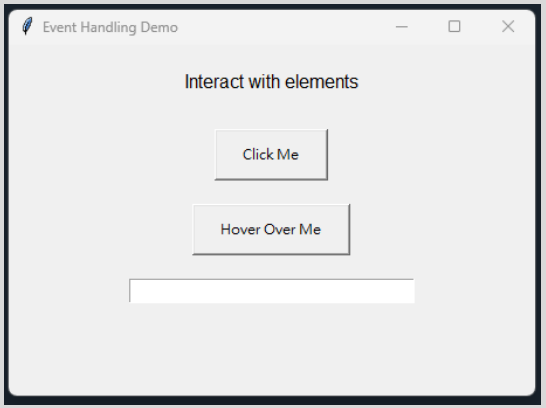
# grid() Advanced Features

- sticky Parameter Values
  - N (North) NE (Northeast) E (East)
    NW (Northwest) SE (Southeast)
    W (West) SW (Southwest) S (South)
  - Combinations
    - "nsew" = Fill entire cell
    - "ew" = Fill horizontally
    - "ns" = Fill vertically

# Event Handling

- Responding to User Actions

```
import tkinter as tk
def on_button_click(event=None):
        """Handle button click event"""
        label.config(text="Button Clicked!")
def on_key_press(event):
        """Handle keyboard event"""
        label.config(text=f"Key pressed: {event.char}")
def on_mouse_enter(event):
        """Handle mouse enter event"""
        event.widget.config(bg="lightblue")
def on_mouse_leave(event):
        """Handle mouse leave event"""
        event.widget.config(bg="SystemButtonFace")

root = tk.Tk()
root.title("Event Handling Demo")
root.geometry("450x300")
```



# Event Handling

- Responding to User Actions

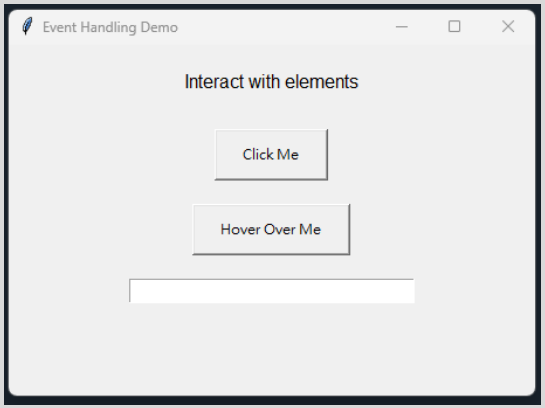
```
# Status label
label = tk.Label(
        root,
        text="Interact with elements",
        font=("Arial", 12),
        pady=20
)
label.pack()
# Button with command
btn = tk.Button(
        root,
        text="Click Me",
        command=on_button_click,
        padx=20,
        pady=10
)
btn.pack(pady=10)
```

```
# Button with hover effects
hover_btn = tk.Button(
        root,
        text="Hover Over Me",
        padx=20,
        pady=10
)

hover_btn.pack(pady=10)
hover_btn.bind("<Enter>", on_mouse_enter)
hover_btn.bind("<Leave>", on_mouse_leave)
```

# Event Handling

- Responding to User Actions

```
# Entry with keyboard events
entry = tk.Entry(root, font=("Arial", 11), width=30)
entry.pack(pady=10)
entry.bind("<KeyPress>", on_key_press)
entry.bind("<Return>", on_button_click)          # Enter key

root.mainloop()
```



# Common Event Types

- Mouse and Keyboard Events

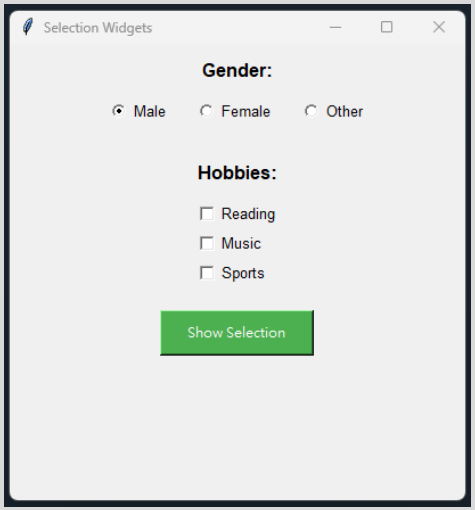
```
# Mouse Events
<Button-1>          # Left mouse button click
<Button-2>          # Middle mouse button click
<Button-3>          # Right mouse button click
<Double-Button-1>   # Double click
<ButtonRelease-1>   # Mouse button release
<Motion>            # Mouse movement
<Enter>             # Mouse enters widget
<Leave>             # Mouse leaves widget

# Keyboard Events
<KeyPress>          # Any key press
<KeyRelease>        # Key release
<Return>            # Enter key
<Tab>               # Tab key
<Escape>            # Escape key
<space>             # Spacebar
<Up>, <Down>        # Arrow keys
<Control-c>         # Control+C
<Alt-F4>            # Alt+F4

# Window Events
<Configure>   # Window resized
<Destroy>     # Widget destroyed
<FocusIn>     # Widget gets focus
<FocusOut>    # Widget loses focus
```

# Tkinter Advanced Topics

# Checkbutton and Radiobutton

- Multiple Choice Widgets

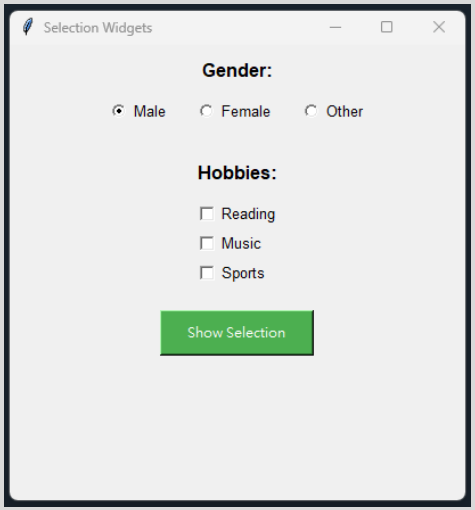
```
import tkinter as tk
def show_selection():
    """Display selected options"""
    hobbies = []
    if var_reading.get():
        hobbies.append("Reading")
    if var_music.get():
        hobbies.append("Music")
    if var_sports.get():
        hobbies.append("Sports")

    gender = gender_var.get()

    result = f"Gender: {gender}\nHobbies: {', '.join(hobbies)}"
    result_label.config(text=result)

root = tk.Tk()
root.title("Selection Widgets")
root.geometry("400x400")
```



# Checkbutton and Radiobutton

- Multiple Choice Widgets

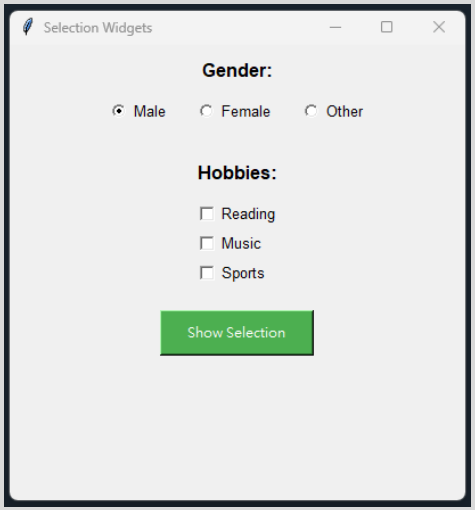
```
# Radiobuttons for gender
tk.Label(root, text="Gender:", font=("Arial", 12, "bold")).pack(pady=10)
gender_var = tk.StringVar(value="Male") # Default value
radio_frame = tk.Frame(root)
radio_frame.pack()
tk.Radiobutton(
    radio_frame,
    text="Male",
    variable=gender_var,
    value="Male",
    font=("Arial", 10) ).pack(side=tk.LEFT, padx=10)
tk.Radiobutton(
    radio_frame,
    text="Female",
    variable=gender_var,
    value="Female",
    font=("Arial", 10) ).pack(side=tk.LEFT, padx=10)
tk.Radiobutton(
    radio_frame,
    text="Other",
    variable=gender_var,
    value="Other",
    font=("Arial", 10) ).pack(side=tk.LEFT, padx=10)
```



# Checkbutton and Radiobutton

- Multiple Choice Widgets

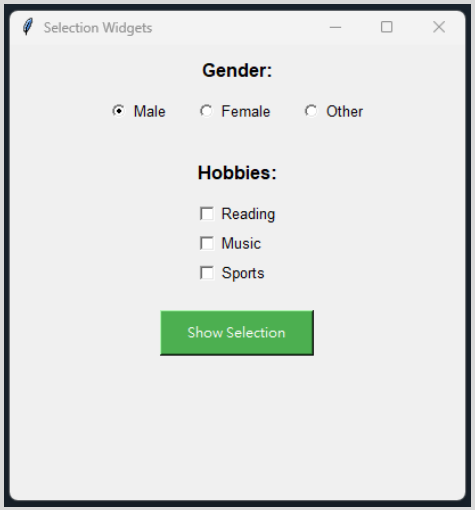
```
# Checkbuttons for hobbies
tk.Label(root, text="\nHobbies:", font=("Arial", 12, "bold")).pack(pady=10)
var_reading = tk.BooleanVar()
var_music = tk.BooleanVar()
var_sports = tk.BooleanVar()
hobby_frame = tk.Frame(root)
hobby_frame.pack()
tk.Checkbutton(
    hobby_frame,
    text="Reading",
    variable=var_reading,
    font=("Arial", 10) ).pack(anchor=tk.W)
tk.Checkbutton(
    hobby_frame,
    text="Music",
    variable=var_music,
    font=("Arial", 10) ).pack(anchor=tk.W)
tk.Checkbutton(
    hobby_frame,
    text="Sports",
    variable=var_sports,
    font=("Arial", 10) ).pack(anchor=tk.W)
```



# Checkbutton and Radiobutton

- Multiple Choice Widgets

```
# Submit button
tk.Button(
    root,
    text="Show Selection",
    command=show_selection,
    bg="#4CAF50",
    fg="white",
    padx=20,
    pady=8
).pack(pady=20)

# Result label
result_label = tk.Label(
    root,
    text="",
    font=("Arial", 10),
    fg="blue"
)
result_label.pack(pady=10)

root.mainloop()
```



# Listbox and Combobox

- Selection Widgets

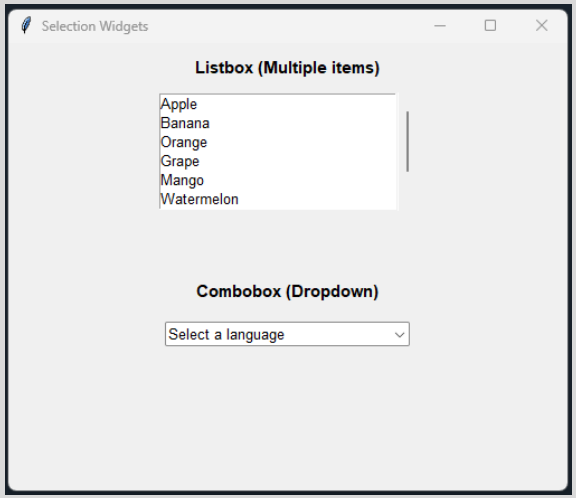
```
import tkinter as tk
from tkinter import ttk

def on_listbox_select(event):
    """Handle listbox selection"""
    selection = listbox.curselection()
    if selection:
        index = selection[0]
        value = listbox.get(index)
        label1.config(text=f"Listbox: {value}")

def on_combobox_select(event):
    """Handle combobox selection"""
    value = combo.get()
    label2.config(text=f"Combobox: {value}")

root = tk.Tk()
root.title("Selection Widgets")
root.geometry("500x400")
```

```
# Listbox
tk.Label(root, text="Listbox (Multiple items)", font=("Arial", 11, "bold")).pack(pady=10)

listbox_frame = tk.Frame(root)
listbox_frame.pack()

scrollbar = tk.Scrollbar(listbox_frame)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

listbox = tk.Listbox(
    listbox_frame,
    height=6,
    width=30,
    font=("Arial", 10), yscrollcommand=scrollbar.set
)
listbox.pack(side=tk.LEFT)
scrollbar.config(command=listbox.yview)
```

# Listbox and Combobox

- Selection Widgets

```
# Add items to listbox
fruits = ["Apple", "Banana", "Orange", "Grape", "Mango", "Watermelon", "Strawberry ", "Pineapple "]
for fruit in fruits:
    listbox.insert(tk.END, fruit)

listbox.bind("<<ListboxSelect>>", on_listbox_select)

label1 = tk.Label(root, text="", font=("Arial", 10), fg="green")
label1.pack(pady=5)

# Combobox
tk.Label(root, text="\nCombobox (Dropdown)", font=("Arial", 11, "bold")).pack(pady=10)

combo = ttk.Combobox(
    root,
    values=["Python", "Java", "JavaScript", "C++", "C#", "Ruby"],
    font=("Arial", 10),
    width=28,
    state="readonly"        # Read-only
)
combo.pack(pady=5)
combo.set("Select a language")      # Default text
combo.bind("<<ComboboxSelected>>", on_combobox_select)

label2 = tk.Label(root, text="", font=("Arial", 10), fg="blue")
label2.pack(pady=5)

root.mainloop()
```



# Menus and Menubar

- Creating Application Menus

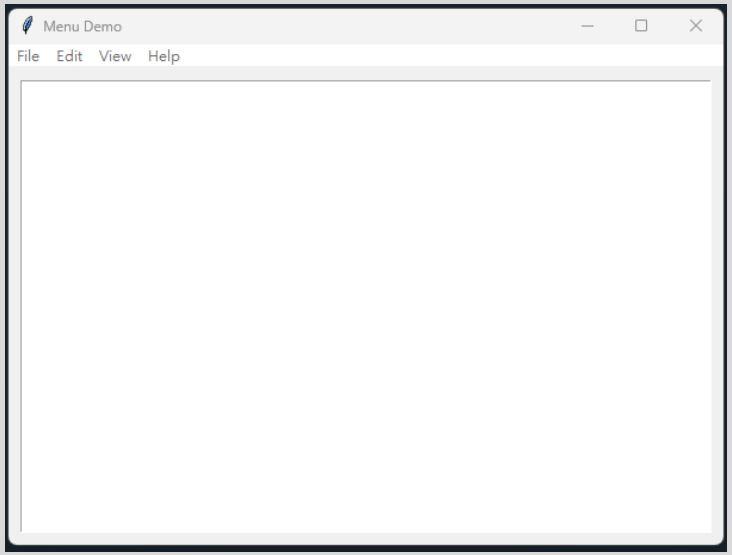
```
import tkinter as tk
from tkinter import messagebox


def new_file():
    """Create new file"""
    messagebox.showinfo("New", "Creating new file... ")
def open_file():
    """Open file"""
    messagebox.showinfo("Open", "Opening file...")
def save_file():
    """Save file"""
    messagebox.showinfo("Save", "Saving file...")
def exit_app():
    """Exit application""“
     if messagebox.askokcancel("Exit", "Do you want to exit?"):
          root.destroy()
def show_about():
    """Show about dialog"""
    messagebox.showinfo(
          "About",
          "GUI Demo Application\nVersion 1.0"
    )
```

```
root = tk.Tk()
root.title("Menu Demo")
root.geometry("600x400")
# Create menubar
menubar = tk.Menu(root)
root.config(menu=menubar)
# File menu
file_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="File", menu=file_menu)
file_menu.add_command(label="New", command=new_file, accelerator="Ctrl+N")
file_menu.add_command(label="Open", command=open_file, accelerator="Ctrl+O")
file_menu.add_command(label="Save", command=save_file, accelerator="Ctrl+S")
file_menu.add_separator() # Separator line
file_menu.add_command(label="Exit", command=exit_app, accelerator="Alt+F4")
# Edit menu
edit_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="Edit", menu=edit_menu)
edit_menu.add_command(label="Cut", accelerator="Ctrl+X")
edit_menu.add_command(label="Copy", accelerator="Ctrl+C")
edit_menu.add_command(label="Paste", accelerator="Ctrl+V")
```

# Menus and Menubar

- Creating Application Menus

```
# View menu with checkbuttons
view_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="View", menu=view_menu)

show_toolbar = tk.BooleanVar(value=True)
show_statusbar = tk.BooleanVar(value=True)

view_menu.add_checkbutton(label="Toolbar", variable=show_toolbar)
view_menu.add_checkbutton(label="Status Bar", variable=show_statusbar)
# Help menu
help_menu = tk.Menu(menubar, tearoff=0)
 menubar.add_cascade(label="Help", menu=help_menu)
help_menu.add_command(label="Documentation")
help_menu.add_separator()
help_menu.add_command(label="About", command=show_about)
# Main content area
text_area = tk.Text(
        root,
        font=("Consolas", 11),
        wrap=tk.WORD
)
text_area.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

root.mainloop()
```



# Dialog Boxes

- Common Dialog Types

```
import tkinter as tk
from tkinter import messagebox, filedialog, colorchooser, simpledialog


def show_info():
    """Information dialog"""
    messagebox.showinfo("Info", "This is an information message")
def show_warning():
    """Warning dialog"""
    messagebox.showwarning("Warning", "This is a warning!")
def show_error():
    """Error dialog"""
    messagebox.showerror("Error", "An error occurred!")
def ask_question():
    """Question dialog"""
    result = messagebox.askyesno("Question", "Do you like Python?")
    if result:
        result_label.config(text="You clicked Yes!")
    else:
        result_label.config(text="You clicked No!")
def ask_ok_cancel():
    """OK/Cancel dialog"""
    result = messagebox.askokcancel("Confirm", "Continue?")
    result_label.config(text=f"Result: {result}")
```

```
def open_file_dialog():
    """File open dialog"""
    filename = filedialog.askopenfilename(
        title="Select a file",
        filetypes=[
            ("Text files", "*.txt"),
            ("Python files", "*.py"),
            ("All files", "*.*")
        ]
    )
    if filename:
        result_label.config(text=f"Selected: {filename}")
def save_file_dialog():
    """File save dialog"""
    filename = filedialog.asksaveasfilename(
        title="Save file",
        defaultextension=".txt",
        filetypes=[("Text files", "*.txt"), ("All files", "*.*")]
    )
    if filename:
        result_label.config(text=f"Save to: {filename}")
```

# Dialog Boxes

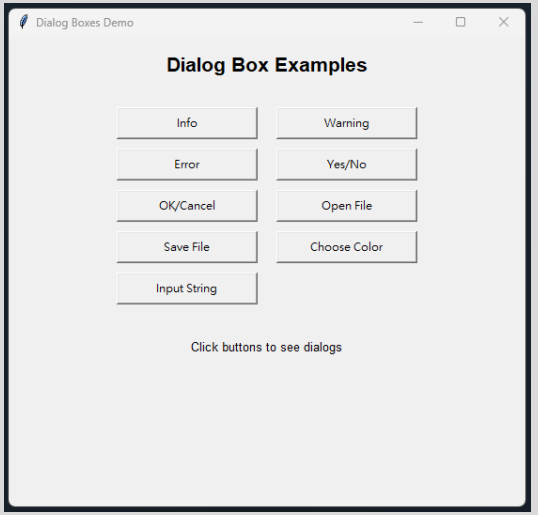
- Common Dialog Types

```
def choose_color():
    """Color chooser dialog"""
    color = colorchooser.askcolor(title="Choose color")
    if color[1]:
        result_label.config(
            text=f"Selected color: {color[1]}", bg=color[1]
        )
def ask_string():
    """String input dialog"""
    name = simpledialog.askstring("Input", "What is your name?")
    if name:
        result_label.config(text=f"Hello, {name}!")

root = tk.Tk()
root.title("Dialog Boxes Demo")
root.geometry("550x500")

# Title
tk.Label(
    root,
    text="Dialog Box Examples",
    font=("Arial", 16, "bold")
).pack(pady=15)

# Button frame
button_frame = tk.Frame(root)
button_frame.pack(pady=10)
# Create buttons for each dialog type
dialogs = [
    ("Info", show_info),
    ("Warning", show_warning),
    ("Error", show_error),
    ("Yes/No", ask_question),
    ("OK/Cancel", ask_ok_cancel),
    ("Open File", open_file_dialog),
    ("Save File", save_file_dialog),
    ("Choose Color", choose_color),
    ("Input String", ask_string)
]
for i, (text, command) in enumerate(dialogs):
    btn = tk.Button(
        button_frame,
        text=text,
        command=command,
        width=20,
        pady=5
    )
    btn.grid(row=i//2, column=i%2, padx=10, pady=5)
```

# Dialog Boxes

- Common Dialog Types

```
# Result label
result_label = tk.Label(
    root,
    text="Click buttons to see dialogs",
    font=("Arial", 10),
    wraplength=400,
    pady=20
)
result_label.pack()

root.mainloop()
```



# Canvas Widget

- Drawing and Graphics

```
import tkinter as tk
def draw_shapes():
    """Draw various shapes on canvas"""
    # Rectangle
    canvas.create_rectangle(
        50, 50, 150, 100,
        fill="lightblue",
        outline="blue",
        width=2
    )
    canvas.create_text(100, 75, text="Rectangle")

    # Oval
    canvas.create_oval(
        200, 50, 300, 150,
        fill="lightgreen",
        outline="green",
        width=2
    )
    canvas.create_text(250, 100, text="Oval")

    # Line
    canvas.create_line(
        350, 50, 450, 150,
        fill="red",
        width=3,
        arrow=tk.LAST # Arrow at end
    )
    canvas.create_text(400, 160, text="Line")

    # Polygon
    points = [50, 200, 100, 280, 150, 200, 100, 220]
    canvas.create_polygon(
        points,
        fill="yellow",
        outline="orange",
        width=2
    )
    canvas.create_text(100, 300, text="Polygon 多邊形")
```

# Canvas Widget

- Drawing and Graphics

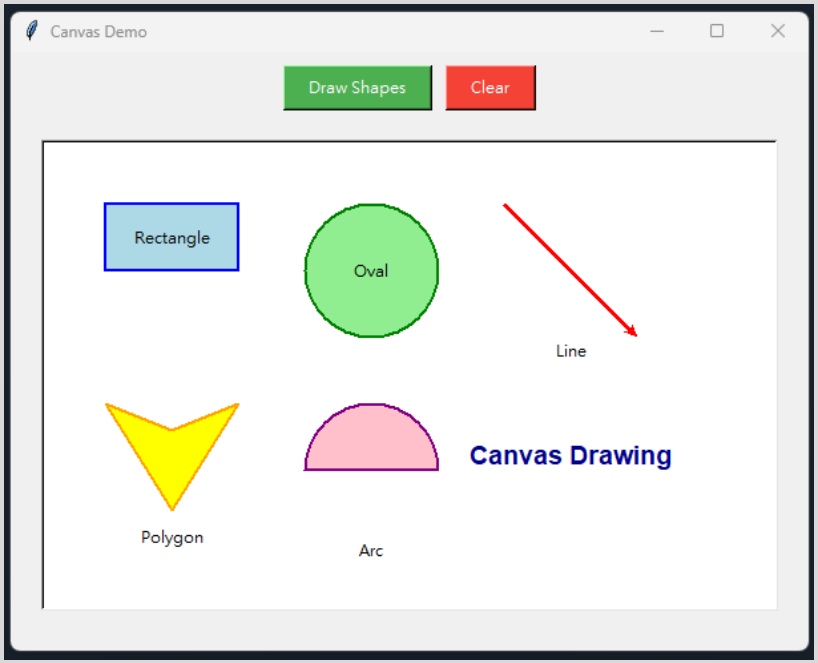
```
    # Arc
    canvas.create_arc(
        200, 200, 300, 300,
        start=0,
        extent=180,
        fill="pink",
        outline="purple",
        width=2
    )
    canvas.create_text(250, 310, text="Arc")

    # Text
    canvas.create_text(
        400, 250,
        text="Canvas Drawing\n",
        font=("Arial", 14, "bold"),
        fill="darkblue"
    )

def clear_canvas():
    """Clear all drawings""“
    canvas.delete("all")
```

```
root = tk.Tk()
root.title("Canvas Demo")
root.geometry("600x450")

# Button frame
button_frame = tk.Frame(root)
button_frame.pack(pady=10)
tk.Button(
    button_
    frame,
    text="Draw Shapes",
    command=draw_shapes,
    bg="#4CAF50",
    fg="white",
    padx=15,
    pady=5
).pack(side=tk.LEFT, padx=5)
tk.Button(
    button_frame,
    text="Clear",
    command=clear_canvas,
    bg="#f44336",
    fg="white",
    padx=15,
    pady=5
).pack(side=tk.LEFT, padx=5)
```

# Canvas Widget

- Drawing and Graphics

```
# Canvas
canvas = tk.Canvas(
    root,
    width=550,
    height=350,
    bg="white",
    relief=tk.SUNKEN,
    bd=2
)
canvas.pack(padx=10, pady=10)

root.mainloop()
```



# PyQt/PySide Introduction

# What is PySide?

- Official Qt for Python
- PySide Features
  - Official Qt Binding
    - Maintained by The Qt Company
    - Full Qt framework support
    - Free LGPL license
  - Cross-Platform
    - Windows, macOS, Linux, iOS, Android
  - Rich Widget Set
    - 1000+ classes and 6000+ functions
    - Professional appearance
  - Qt Designer
    - Visual GUI design
    - Drag-and-drop interface
  - Powerful Capabilities
    - Database integration
    - Network programming
    - Multimedia support

# PySide vs Tkinter

- Comparison

| Feature | Tkinter | PySide |
|---|---|---|
| Learning Curve | Easy | Moderate |
| Appearance | Basic | Professional |
| Installation | Built-in | Requires install |
| Documentation | Good | Excellent |
| Performance | Good | Excellent |
| Advanced Features | Limited | Extensive |
| License | Free | LGPL (Free) |
| Official Support | Python Foundation | The Qt Company |
| Best For | Simple apps | Professional apps |

- Key Advantage
  - PySide is completely free with LGPL license, making it ideal for both commercial and open-source projects!

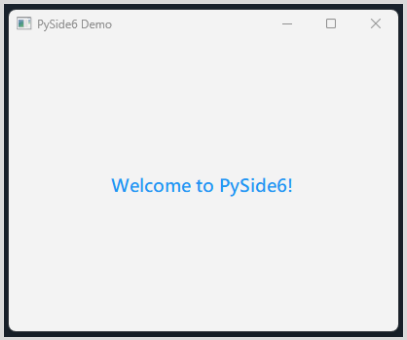
# Installing PySide

- Installation Steps
  - <Conda>
  - # Install PySide6 (Recommended - Latest)<br>pip install PySide6
  - # Or install PySide2 (For Qt 5.x)<br>pip install PySide2
- PySide Versions
  - PySide6: For Qt 6.x, recommended for new projects
  - PySide2: For Qt 5.x, more stable and mature
  - 100% API Compatible: Code is almost identical between PySide and PyQt
- Why PySide?
  - Official Qt binding
  - LGPL license - Free for commercial use
  - Same API as PyQt
  - Better long-term support

# First PySide Application

- Basic Window Structure

```
import sys
from PySide6.QtWidgets import QApplication, QMainWindow, QLabel
from PySide6.QtCore import Qt

class MainWindow(QMainWindow):
    """Main application window"""
    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        """Initialize user interface"""
        # Set window properties
        self.setWindowTitle('PySide6 Demo')
        self.setGeometry(100, 100, 400, 300)    # x, y, width, height
        # Create central widget
        label = QLabel('Welcome to PySide6!', self)
        label.setAlignment(Qt.AlignCenter)      # Center alignment
        label.setStyleSheet("""
            QLabel {
                font-size: 18px;
                font-weight: bold;
                color: #2196F3;
            }
        """)
        # Set as central widget
        self.setCentralWidget(label)

def main():
    """Main function"""
    # Create application
    app = QApplication(sys.argv)

    # Create and show main window
    window = MainWindow()
    window.show()

    # Start event loop
    sys.exit(app.exec())
    # Note: exec() in PySide6, exec_() in PySide2

if __name__ == '__main__':
    main()
```



# PySide Basic Widgets

- Common Widgets

```
import sys
from PySide6.QtWidgets import (
    QApplication, QMainWindow, QWidget, QVBoxLayout,
    QLabel, QPushButton, QLineEdit, QTextEdit, QMessageBox
)

class WidgetDemo(QMainWindow):
    """Widget demonstration"""
    def __init__(self):
        super().__init__()
        self.init_ui()
    def init_ui(self):
        """Initialize UI"""
        self.setWindowTitle('PySide Widgets')
        self.setGeometry(100, 100, 500, 400)

        # Central widget
        central_widget = QWidget()
        self.setCentralWidget(central_widget)

        # Layout
        layout = QVBoxLayout()
        central_widget.setLayout(layout)
```

```
        # Label
        label = QLabel('Enter your name:', self)
        label.setStyleSheet("font-size: 14px; font-weight: bold;")
        layout.addWidget(label)

        # Line Edit
        self.name_input = QLineEdit(self)
        self.name_input.setPlaceholderText('Type here...')
        self.name_input.setStyleSheet("""
            QLineEdit {
                padding: 8px;
                font-size: 12px;
                border: 2px solid #ddd;
                border-radius: 4px;
            }
            QLineEdit:focus {
                border-color: #2196F3;
            }
        """)
        layout.addWidget(self.name_input)
```

# PySide Basic Widgets

- Common Widgets

```
# Button
button = QPushButton('Greet Me', self)
button.setStyleSheet("""
    QPushButton {
        background-color: #4CAF50;
        color: white;
        padding: 10px;
        font-size: 14px;
        border: none;
        border-radius: 4px;
    }
    QPushButton:hover {
        background-color: #45a049;
    }
    QPushButton:pressed {
        background-color: #3d8b40;
    }
""")
button.clicked.connect(self.on_button_click)
layout.addWidget(button)
```

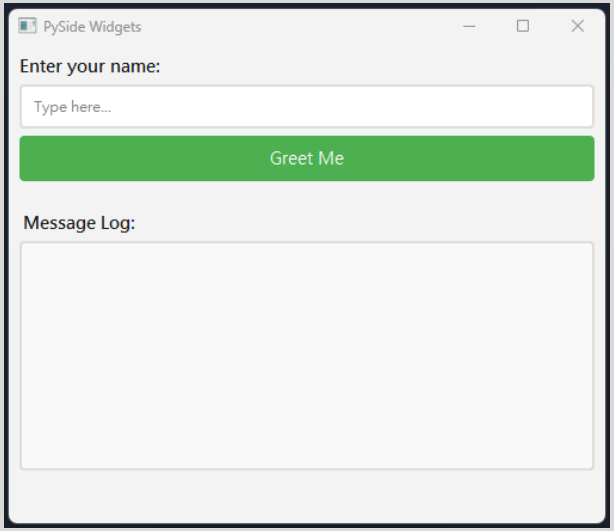
```
# Text Edit
label2 = QLabel('Message Log:', self)
label2.setStyleSheet("font-size: 14px; font-weight: bold;
margin-top: 20px;")
layout.addWidget(label2)

self.text_edit = QTextEdit(self)
self.text_edit.setReadOnly(True)
self.text_edit.setStyleSheet("""
    QTextEdit {
        font-size: 12px;
        border: 2px solid #ddd;
        border-radius: 4px;
        background-color: #f9f9f9;
    }
""")
layout.addWidget(self.text_edit)

# Add stretch to push widgets to top
layout.addStretch()
```

# PySide Basic Widgets

- Common Widgets

```
def on_button_click(self):
    """Handle button click"""
    name = self.name_input.text()

    if name.strip():
        # Add message to log
        message = f"Hello, {name}!“
        self.text_edit.append(message)

        # Show message box
        QMessageBox.information(
            self,
            'Greeting’,
            message,
            QMessageBox.Ok
        )

        # Clear input
        self.name_input.clear()
    else:
        # Show warning
        QMessageBox.warning(
            self,
            'Warning’,
            'Please enter your name!’,
            QMessageBox.Ok
        )
```

```
def main():
    app = QApplication(sys.argv)
    window = WidgetDemo()
    window.show()
    sys.exit(app.exec())

if __name__ == '__main__’:
    main()
```



# PySide Layouts

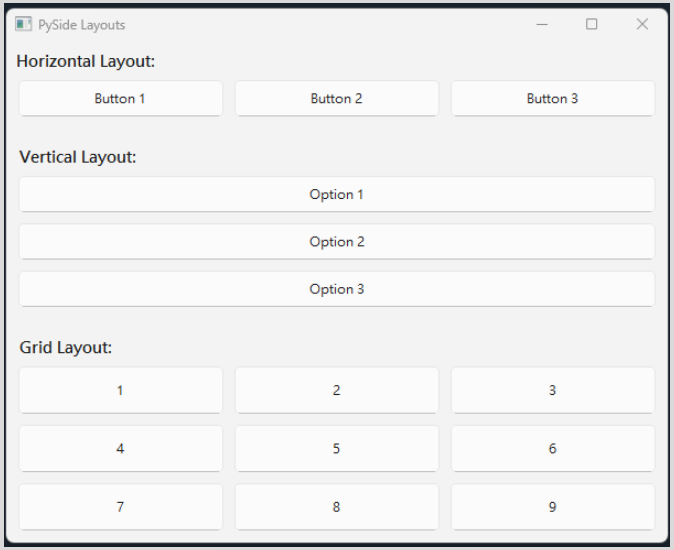
- Layout Managers

```
import sys
from PySide6.QtWidgets import (
    QApplication, QMainWindow, QWidget,
    QVBoxLayout, QHBoxLayout, QGridLayout,
    QPushButton, QLabel
)

class LayoutDemo(QMainWindow):
    """Layout demonstration"""

    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        """Initialize UI"""
        self.setWindowTitle('PySide Layouts')
        self.setGeometry(100, 100, 600, 400)

        # Central widget
        central_widget = QWidget()
        self.setCentralWidget(central_widget)

        # Main vertical layout
        main_layout = QVBoxLayout()
        central_widget.setLayout(main_layout)

        # Horizontal Layout Example
        hlayout_label = QLabel('Horizontal Layout:')
        hlayout_label.setStyleSheet("font-weight: bold; font-size: 14px;")
        main_layout.addWidget(hlayout_label)

        hlayout = QHBoxLayout()
        for i in range(1, 4):
            btn = QPushButton(f'Button {i}')
            btn.setStyleSheet("padding: 10px;")
            hlayout.addWidget(btn)
        main_layout.addLayout(hlayout)

        # Vertical Layout Example
        vlayout_label = QLabel('Vertical Layout:')
        vlayout_label.setStyleSheet("font-weight: bold; font-size: 14px;
margin-top: 20px;")
        main_layout.addWidget(vlayout_label)

        vlayout = QVBoxLayout()
        for i in range(1, 4):
            btn = QPushButton(f'Option {i}')
            btn.setStyleSheet("padding: 10px;")
            vlayout.addWidget(btn)
        main_layout.addLayout(vlayout)
```

# PySide Layouts

- Layout Managers

```
        # Grid Layout Example
        glayout_label = QLabel('Grid Layout:')
        glayout_label.setStyleSheet("font-weight: bold; font-size: 14px; margin-top: 20px;")
        main_layout.addWidget(glayout_label)

        glayout = QGridLayout()
        positions = [(i, j) for i in range(3) for j in range(3)]
        for i, position in enumerate(positions):
            btn = QPushButton(f'{i+1}')
            btn.setStyleSheet("padding: 15px;")
            glayout.addWidget(btn, *position)
        main_layout.addLayout(glayout)

        main_layout.addStretch()

def main():
    app = QApplication(sys.argv)
    window = LayoutDemo()
    window.show()
    sys.exit(app.exec())

if __name__ == '__main__':
    main()
```



# Signals and Slots

- Event Handling Mechanism
- Concept
  - Signal: Event emitted by widget
  - Slot: Function that responds to signal
  - Connection: Link between signal and slot

```
import sys
from PySide6.QtWidgets import (
    QApplication, QMainWindow, QWidget,
    QVBoxLayout, QPushButton, QLabel, QSlider
)
from PySide6.QtCore import Qt

class SignalSlotDemo(QMainWindow):
    """Signal and slot demonstration"""
    def __init__(self):
    super().__init__()
    self.init_ui()

    def init_ui(self):
        """Initialize UI"""
        self.setWindowTitle('Signals & Slots')
        self.setGeometry(100, 100, 450, 300)

        central_widget = QWidget()
        self.setCentralWidget(central_widget)
        layout = QVBoxLayout()
        central_widget.setLayout(layout)
        # Button with click signal
        self.click_count = 0
        self.click_label = QLabel('Button clicks: 0')
        self.click_label.setStyleSheet("font-size: 14px; padding: 10px;")
        layout.addWidget(self.click_label)
```

# Signals and Slots

```
        click_btn = QPushButton('Click Me!')
        click_btn.setStyleSheet("""
            QPushButton {
                background-color: #2196F3;
                color: white;
                padding: 10px;
                font-size: 14px;
                border-radius: 4px;
            }
            QPushButton:hover {
                background-color: #1976D2;
            }
        """)
        # Connect signal to slot
        click_btn.clicked.connect(self.on_button_clicked)
        layout.addWidget(click_btn)

        # Slider with value changed signal
        layout.addWidget(QLabel('Slider Value:', styleSheet="font-size: 14px; margin-top: 20px;"))

        self.slider_label = QLabel('Value: 50')
        self.slider_label.setStyleSheet("font-size: 12px; color: #4CAF50;")
        layout.addWidget(self.slider_label)
        slider = QSlider(Qt.Horizontal) slider.setMinimum(0)
        slider.setMaximum(100) slider.setValue(50)
        slider.setTickPosition(QSlider.TicksBelow)
        slider.setTickInterval(10)
        # Connect signal to slot
        slider.valueChanged.connect(self.on_slider_changed)
        layout.addWidget(slider)
        layout.addStretch()

    def on_button_clicked(self):
        """Handle button click signal"""
        self.click_count += 1
        self.click_label.setText(
            f'Button clicks: {self.click_count}' )

    def on_slider_changed(self, value):
        """Handle slider value change signal"""
        self.slider_label.setText(f'Value: {value}')

def main():
    app = QApplication(sys.argv)
    window = SignalSlotDemo()
    window.show()
    sys.exit(app.exec())
if __name__ == '__main__':
    main()
```

# Qt Designer Overview

- Visual GUI Design Tool
- Features
  - Drag-and-Drop
    - Add widgets visually
    - Arrange layouts easily
  - Property Editor
    - Configure widget properties
    - Set styles and appearance
  - Signal-Slot Editor
    - Connect events visually
    - Define custom connections
  - Preview Function
    - Test UI before coding
    - See real-time changes

# Using Qt Designer Files

- Loading .ui Files

```
import sys
from PySide6.QtWidgets
import QApplication, QMainWindow
from PySide6.QtUiTools import QUiLoader
from PySide6.QtCore import QFile

class MainWindow(QMainWindow):
    """Load UI from .ui file"""
    def __init__(self):
    super().__init__()
    # Load UI file using QUiLoader
    ui_file = QFile("mainwindow.ui")
    ui_file.open(QFile.ReadOnly)

    loader = QUiLoader()
    self.ui = loader.load(ui_file)
    ui_file.close()
    # Access widgets defined in Qt Designer
    # self.ui.pushButton.clicked.connect(self.on_button_click)

def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.ui.show()
    sys.exit(app.exec())

if __name__ == '__main__’:
    main()
```

# Using Qt Designer Files

- Converting .ui to .py
  - # Convert using pyside6-uic
    pyside6-uic mainwindow.ui -o mainwindow.py
  - # For PySide2
    pyside2-uic mainwindow.ui -o mainwindow.py
  - # Then import in your code
    from mainwindow import Ui_MainWindow
- Note
  - Qt Designer files (.ui) are compatible with both PyQt and PySide. Just use the appropriate conversion tool.